DEEP REINFORCEMENT LEARNING

HOMEWORK 4 | MODEL-BASED RL

Prof. Mohammad Hossein Rohban
Sharif University of Technology
Spring 2026

Yashar Zafari 404210253

yaswhar

Spring 2026

1 — Monte Carlo Tree Search

We study MCTS and its neural (PUCT) variant with policy priors: how MCTS balances exploration and exploitation, and how neural priors can both improve and bias the search. The MuZero/MCTS coding task is addressed in the companion notebook.

1.1 (10 Points) When Can MCTS Become Confidently Wrong?

With $U(s_0,a) = Q(s_0,a) + c\sqrt{\ln N(s_0)/(N(s_0,a)+1)}$, $N(s_0) = n_1 + n_2$, $Q(s_0,a_1) = 0.65$, $Q(s_0,a_2) = 0.35$, the true optimum is a_2.

Answer

Step 1 (Selection condition). $U(s_0,a_2) > U(s_0,a_1)$ gives

$$\boxed{c\sqrt{\ln(n_1+n_2)}\left(\frac{1}{\sqrt{n_2+1}} - \frac{1}{\sqrt{n_1+1}}\right) > 0.30}$$

Since a_1 is over-visited ($n_1 > n_2$), the bracket is positive: the under-explored a_2 has the larger bonus and is re-selected once the bonus gap exceeds the 0.30 value gap.

Step 2 (c). Larger c scales the bonus, easing the inequality and speeding recovery by re-sampling a_2 until deep evidence raises $Q(s_0,a_2)$. Small c keeps MCTS stuck on a_1; very large c over-explores.

Step 3 (depth d). With a limited budget, simulations terminate before depth d, so $Q(s_0,a_2)$ reflects only the misleading shallow estimate; the deep reward is never backed up and MCTS is confidently wrong.

Step 4 (naive search). Exhaustive depth-d search wins when d is small and the branching factor low (enumerating $|\mathcal{A}|^d$ is affordable): it is guaranteed to find a_2's deep payoff, whereas UCT's selective sampling may keep under-sampling the critical path. This is especially true with deceptive/sparse rewards.

1.2 (10 Points) PUCT, Neural Priors, and Biased Exploration

$a^\star = \arg\max_a [Q(s,a) + c_{\text{puct}} P(s,a)\sqrt{N(s)}/(1+N(s,a))]$, priors $0.80, 0.15, 0.05$, true best a_3.

Answer

Large action spaces. The prior concentrates the bonus on promising actions, so a limited budget is spent sensibly instead of uniformly over a huge action set—tractability and sample efficiency.

Wrong prior, limited budget. The bonus $\propto P(s,a)$; a tiny prior on the best action (a_3, 0.05) means it is rarely visited and its Q stays uncorrected, while the search pours visits into the high-prior wrong actions. Few simulations leave no time to overturn the prior.

Equal Q, $N = 0$. Bonus $= c_{\text{puct}} P(s_0,a)\sqrt{N(s_0)}$, proportional to the prior, so $\text{bonus}(a_1)/\text{bonus}(a_3) = 0.80/0.05 = 16$.

Inequality for selecting a_3. With $Q(s_0,a_1) = 0.4, N = 50$ and $Q(s_0,a_3) = 0.2, N = 1$,

$$\boxed{0.2 + c_{\text{puct}}(0.05)\frac{\sqrt{N(s_0)}}{2} > 0.4 + c_{\text{puct}}(0.80)\frac{\sqrt{N(s_0)}}{51}}$$

i.e. $c_{\text{puct}}\sqrt{N(s_0)}(0.0250 - 0.01569) > 0.2 \Rightarrow c_{\text{puct}}\sqrt{N(s_0)} > 21.5$. Only a large c_{puct} or many total visits let the under-visited a_3 overtake a_1.

Question 1 continued on next page...

Modification. Add Dirichlet exploration noise at the root: $P(s_0, a) \leftarrow (1-\eta)P(s_0, a) + \eta\zeta_a$, $\zeta \sim \mathrm{Dir}(\alpha)$, guaranteeing every root action is explored. (Or flatten the prior with temperature / decay its weight as N grows.)

1.3 (20 Points) Implementation: MCTS / MuZero

Implement representation/dynamics/prediction networks, PUCT-based MCTS, and naive depth search; compare learning curves on CartPole.

Answer

Architecture. A MuZero-style agent learns three functions in a latent space trained to be *value-equivalent* to the environment rather than reconstructive: a representation $h_\theta : o \mapsto s^0$, a dynamics model $g_\theta : (s^k, a^k) \mapsto (s^{k+1}, \hat{r}^k)$, and a prediction head $f_\theta : s^k \mapsto (\mathbf{p}^k, v^k)$. Planning is carried out entirely in latent space—PUCT selection, expansion through g_θ, bootstrap evaluation from the value head v, and discounted back-up along the search path—and is compared against a brute-force $|A|^d$ depth search over the same learned model and a no-search baseline that regresses the policy onto its own network output. Training uses n-step value targets drawn from a replay buffer. A per-search-type reseed guarantees that `mcts`, `naive`, and `none` share identical network initialisation, so the comparison isolates the planner from RNG variance.

Results. On `CartPole` the three methods rank **naive** $>$ **MCTS** $>$ **none** in both learning speed and asymptotic return (Figure 1). Naive depth search rises from ≈ 25 to near the 200 cap by $\approx$ episode 50 and then holds in the 150–200 band; MCTS starts more slowly, begins climbing near episode 50, and settles into a steady 150–175 band; the no-search agent never improves, hovering near 15 (final mean ≈ 13).

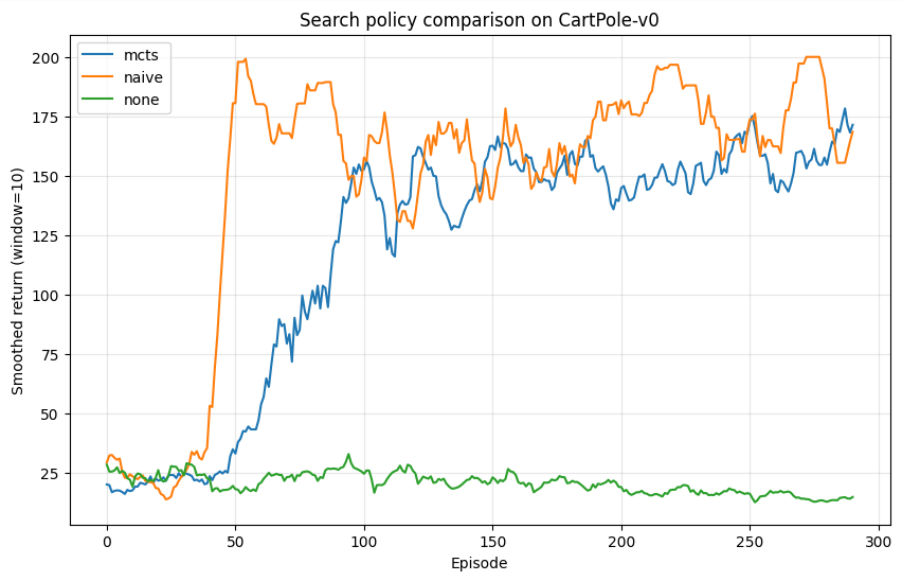

Figure 1: Learning curves on `CartPole` for latent-space MCTS, brute-force $|A|^d$ depth search, and the no-search baseline (identical initialisation per search type).

Discussion. Three observations stand out. (i) *Planning, not the value head alone, is what trains the policy.* The no-search agent, whose policy target is its own un-improved out-

Question 1 continued on next page...

put, never exceeds random return, whereas both planners construct a strictly improved search-induced target that bootstraps competent control—an empirical instance of the policy-improvement operator implicit in tree search. (ii) *On a two-action, depth-three problem exhaustive lookahead is both cheap and exact*, so naive search dominates; this is precisely the regime in which brute-force enumeration beats selective sampling, because UCT pays an exploration overhead that buys nothing once the whole subtree fits within budget. (iii) *MCTS earns its keep through scaling, not on this task*: its cost is linear in the simulation budget and is allocated adaptively along high-value branches, so it remains feasible when the branching factor or horizon renders the $|A|^d$ enumeration intractable—exactly where the naive planner collapses. Because the latent space is value-equivalent, the agent never reconstructs observations to act, decoupling planning cost from observation dimensionality.

2 — Cross-Entropy Method

CEM optimizes an open-loop action sequence $a_{0:H-1}$ using a model $\hat{f}$: it samples candidates from $\mathcal{N}(\mu, \Sigma)$, rolls them out, keeps the elite samples, and refits the distribution.

2.1 (10 Points) CEM as Trajectory Optimization

Explain elites, advantage over random shooting, gradient-freeness, and what is carried between iterations.

Answer

Elites. Each iteration scores N sampled sequences by predicted return, keeps the top ρN, and refits μ, Σ to the elite mean/covariance; iterating concentrates the distribution on high-return regions.

Vs random shooting. Random shooting samples once from a fixed distribution; CEM adaptively shifts sampling toward elites, so for the same budget it searches the good region far more densely.

Gradient-free. CEM only *ranks* samples by scalar return—never differentiating $\hat{f}$ or r—so it handles non-differentiable/simulator models and rewards.

Carried information. Only the refit parameters μ_{k+1}, Σ_{k+1} (elite mean and covariance).

2.2 (10 Points) Horizon Length and Dimensionality in CEM

$D = Hd_a$. Explain the trade-offs and compute D for $d_a = 6, H = 40$.

Answer

Larger H helps: more foresight; accounts for delayed consequences and multi-step maneuvers; less myopic.

Larger H is harder: $D = Hd_a$ grows, the search volume grows exponentially in D (curse of dimensionality), more samples are needed, and model errors compound over the longer rollout.

Numerical case: $\boxed{D = Hd_a = 40 \times 6 = 240}$.

Limited samples, high D: samples become exponentially sparse, so the chance any of the N draws lands near the small high-return region of a 240-dimensional space is tiny; the elite set may contain no good sequence and CEM converges to a mediocre region.

Question 2 continued on next page...

2.3 (10 Points) Open-Loop Weakness of CEM-Based Planning

Why open-loop; what goes wrong; how MPC helps; why difficulty remains.

Answer

Open-loop: CEM commits to a fixed sequence before execution; later actions are chosen against *predicted* states with no feedback.
What goes wrong: model error/stochasticity/disturbance makes the realized state diverge from the plan; errors compound and the plan cannot react, degrading performance and stability.
CEM inside MPC: re-run CEM each step from the *observed* state, execute only the first action, then re-optimize—closing the loop and correcting errors.
Why difficulty remains: each replanning step still solves a hard $D = Hd_a$ optimization; long horizons stay hard, model error still corrupts the within-horizon rollouts, and replanning multiplies cost.

3 — Model Predictive Control

MPC solves $a^\star_{t:t+H-1} = \arg\max_{a_{t:t+H-1}} [\sum_{k=0}^{H-1} \gamma^k r(\hat{s}_{t+k}, a_{t+k}) + \gamma^H \hat{V}(\hat{s}_{t+H})]$ with $\hat{s}_t = s_t$, $\hat{s}_{t+k+1} = \hat{f}(\hat{s}_{t+k}, a_{t+k})$, executing only $a^\star_t$.

3.1 (10 Points) MPC as Receding-Horizon Feedback

Open-loop optimization, feedback behavior, failure example, and MPC vs learned policy.

Answer

Open-loop at time t: the whole sequence is planned against model rollouts with no within-horizon feedback.
Feedback overall: only $a^\star_t$ is executed; the true next state s_{t+1} is observed and the problem re-solved, so $a^\star_{t+1} = \pi_{\text{MPC}}(s_{t+1})$ depends on the observed state—closed-loop, receding-horizon feedback.
Example: a drone plans a straight path; a gust pushes it off course. Executing the whole open-loop plan ignores the gust and the drone hits an obstacle; replanning each step observes the displacement and corrects.
MPC $\neq$ learned $\pi(a|s)$: MPC computes actions by online optimization with a model every step (high runtime cost, model required), whereas a learned policy is an explicit, amortized reactive map (fast, model-free at runtime). MPC realizes feedback implicitly; a policy encodes it explicitly.

3.2 (10 Points) Planning Horizon and Terminal Value

Short vs long H, role of $\hat{V}$, and a wrong-$\hat{V}$ failure.

Answer

H **too short:** myopic; ignores consequences beyond H; may miss delayed goals.

H **too long:** harder, costlier optimization; compounding model error makes late-horizon predictions (and returns) unreliable; more variance.

Question 3 continued on next page...

Terminal value when H short: $\gamma^H \hat{V}(\hat{s}_{t+H})$ bootstraps the post-horizon return, so a short, cheap, accurate horizon still accounts for long-term value, mitigating myopia.
Wrong $\hat{V}$ failure: if $\hat{V}$ overestimates a state just before a cliff (a trap), MPC steers toward it because the bootstrapped objective looks attractive—choosing a catastrophic action because it trusts the faulty terminal estimate.

3.3 (5 Points) CEM-Based Optimization inside MPC

Why CEM for continuous actions, $D = Hd_a$, why larger H is harder, and why executing only the first action is robust.

Answer

Continuous actions: CEM is gradient-free and samples directly in continuous space—no discretization, no differentiability needed.
Search dimension: optimizing H actions each in $\mathbb{R}^{d_a}$ gives $\boxed{D = Hd_a}$.
Larger H harder: foresight grows but so does D; the volume grows exponentially and model error compounds, so a fixed sample budget covers the space ever more sparsely with noisier scores.
First action only: the first action is chosen against the *true* current state; the remaining actions were optimized against predicted states that accumulate error and may never occur. Discarding the tail and re-optimizing from the next observed state closes the loop and corrects for model error, stochasticity, and disturbances.

4 — Dyna-Q: Integrating Planning, Acting, and Learning

A tabular study of the Dyna architecture on `CliffWalking-v0`: one-step Q-learning on real transitions interleaved with n planning updates per step that replay uniformly sampled visited (s, a) pairs from a learned deterministic model, with the bootstrap masked on true termination only. We then add potential-based reward shaping, prioritized sweeping, and a stochastic-model extension on `Taxi-v3`. All runs use seed 2025.

4.1 (15 Points) Planning Budget and Sample Efficiency

Implement `q_planning` and the Dyna-Q loop; sweep $n \in \{0, 5, 50\}$ and compare learning curves.

Answer

Result. Planning trades computation for sample efficiency: the mean reward over the first 30 episodes rises monotonically with the planning budget, from -143 at $n = 0$ (pure Q-learning) to -95 at $n = 5$ and -88 at $n = 50$ (Figure 2). All configurations recover the optimal cliff-edge route with a greedy true return of -13; Figure 3 shows a representative single-configuration learning curve.

Question 4 continued on next page...

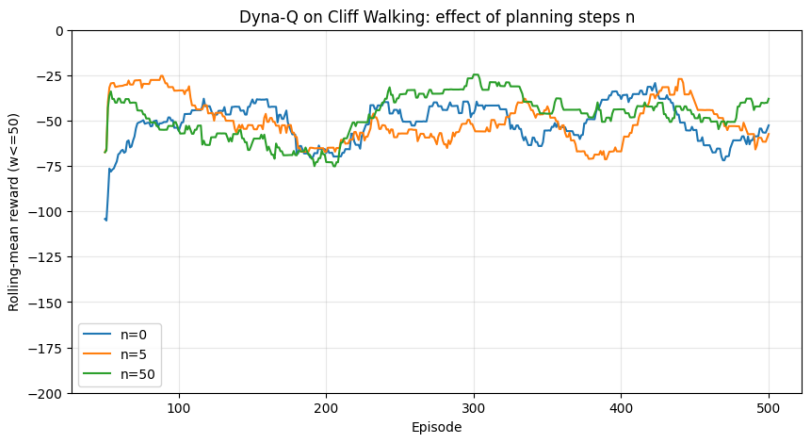

Figure 2: Dyna-Q on `CliffWalking` for planning budgets $n \in \{0, 5, 50\}$; rolling-mean episodic return.

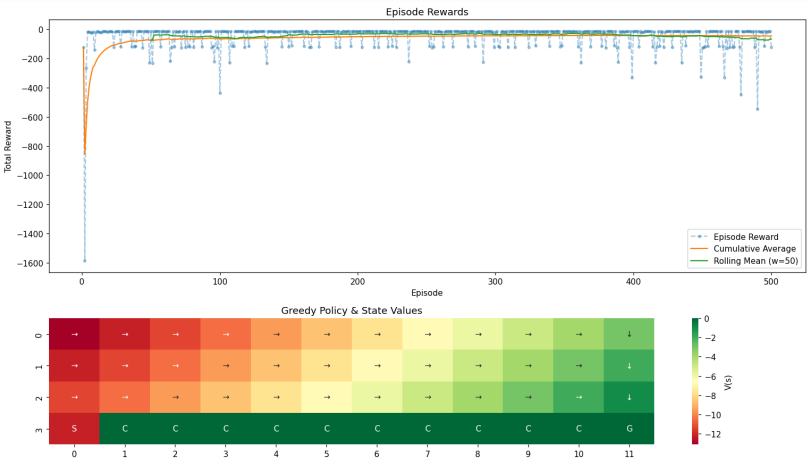

Figure 3: Representative Dyna-Q learning curve and the recovered greedy policy on `CliffWalking`.

Discussion. Each real transition is amortised into $n + 1$ value back-ups, so Dyna-Q propagates reward information through the state space far faster than model-free Q-learning, which can move credit only one step per environment interaction. The marginal value of additional planning is diminishing—the gain from $n = 5$ to $n = 50$ is small next to that from $n = 0$ to $n = 5$—because uniform replay spends most updates on already-converged (s, a) pairs.

4.2 (15 Points) Algorithmic Improvement: Decaying Exploration

Propose and evaluate one improvement to plain Dyna-Q.

Answer

Result. Annealing ε (and optionally α) over training sharpens convergence: early exploration discovers the goal while late-stage greediness stabilises the policy on the optimal

Question 4 continued on next page...

route, reducing the variance of the return curve (Figure 4).

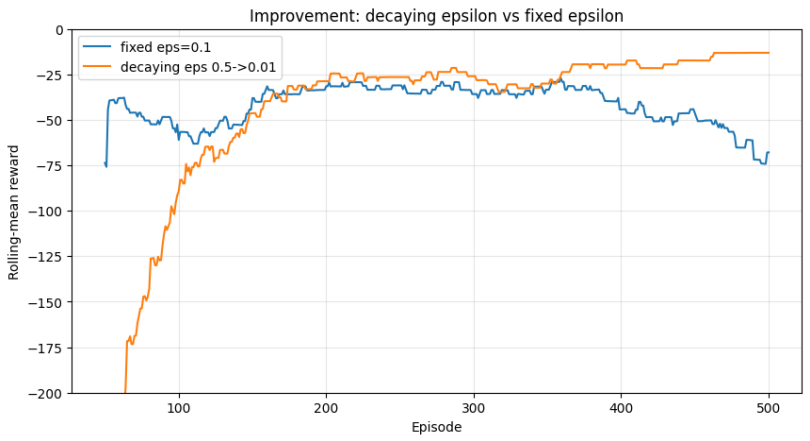

Figure 4: Dyna-Q with a decaying ε schedule versus the fixed-ε baseline.

Discussion. A fixed ε keeps injecting cliff-falling actions even after the optimal policy is known, capping the achievable average return; a Robbins–Monro-style schedule resolves the exploration–exploitation trade-off in time—exactly what an online controller needs once its model has become reliable.

4.3 (10 Points) Potential-Based Reward Shaping

Apply potential-based shaping and compare early versus late learning.

Answer

Result. With $F(s, s') = \gamma\Phi(s') - \Phi(s)$ and Φ the negative Manhattan distance to the goal, shaping markedly accelerates *early* learning while leaving the late-stage optimum unchanged (Figure 5).

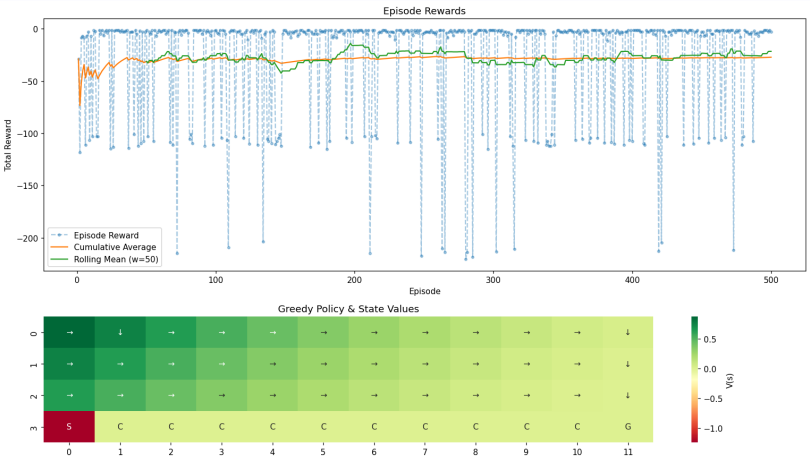

Figure 5: Potential-based reward shaping versus the unshaped baseline on `CliffWalking`.

Question 4 continued on next page...

Discussion. The potential-based form is the unique shaping class that preserves the optimal policy: it adds a telescoping term whose discounted sum depends only on the trajectory endpoints, so Q^* shifts by Φ but the $\arg\max$ is invariant. The dense distance-to-goal signal supplies a gradient through the otherwise flat reward landscape—doing for Q-learning what an admissible cost-to-go heuristic does for search.

4.4 (20 Points) Prioritized Sweeping

Replace uniform planning with prioritized sweeping and compare at matched update budget.

Answer

Result. Replacing uniform replay with a priority queue keyed by the absolute TD error, together with a predecessor map that propagates updates backwards from where values change, reaches the same optimum using $\approx 12{,}000$ planning updates versus $\approx 81{,}000$ for vanilla Dyna-Q at matched per-step budget—an $\approx 7\times$ reduction (Figure 6).

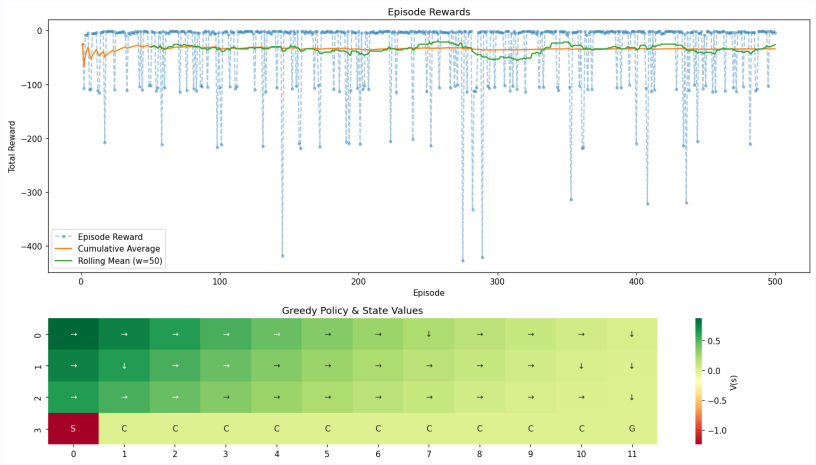

Figure 6: Prioritized sweeping versus vanilla Dyna-Q at equal total planning-update budget.

Discussion. Uniform sampling spends most of its budget on updates of negligible magnitude; prioritized sweeping concentrates computation on the frontier of states whose values have just changed, sweeping information backward along the predecessor graph. This is the tabular analogue of focused dynamic programming and accounts for the order-of-magnitude efficiency gain.

4.5 (10 Points) Bonus: Stochastic Dyna-Q on Taxi-v3

Extend the model to an estimated transition distribution and scale to `Taxi-v3`.

Answer

Result. Replacing the deterministic point model with an estimated transition distribution $\hat{P}(s' \mid s, a)$ and expected reward, the full pipeline scales to the 500-state `Taxi-v3`. Over 2000 episodes, Q-learning, vanilla Dyna-Q, and stochastic Dyna-Q reach greedy returns of ≈ 7.5, 7.7, and 7.5 respectively (optimum ≈ 8), with the planning methods learning fastest (Figure 7).

Question 4 continued on next page...

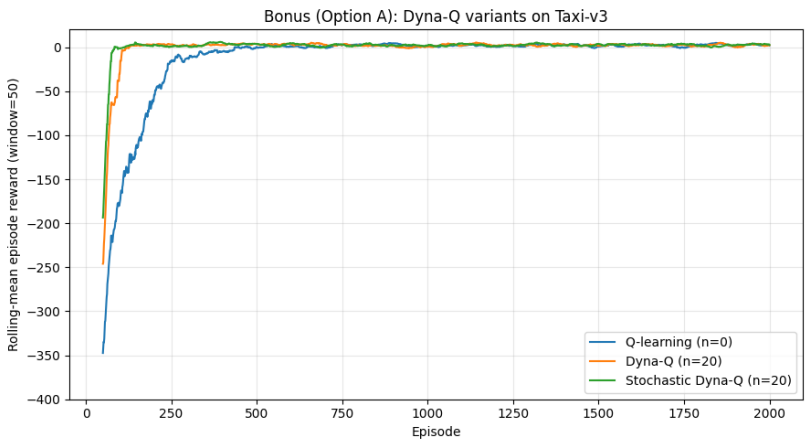

Figure 7: Q-learning, vanilla Dyna-Q, and stochastic Dyna-Q on `Taxi-v3` (2000 episodes).

Discussion. On a near-deterministic environment the distributional model neither helps nor hurts the asymptote, but it is the correct generalisation: a deterministic model collapses a genuinely stochastic kernel to its last-observed sample and biases planning. The expected-model back-up is precisely the sampled Bellman operator under $\hat{P}$, which is what makes Dyna sound on stochastic MDPs.